

Restomate Backend API Documentation

Daftar Isi

- [Overview](#)
- [Setup & Konfigurasi](#)
- [Struktur Database](#)
- [API Endpoints](#)
- [LLM Integration](#)
- [Error Handling](#)
- [Testing](#)

Overview

Restomate adalah sistem informasi berbasis LLM untuk manajemen restoran yang menyediakan analisis data, rekomendasi AI, dan insights bisnis komprehensif.

Fitur Utama:

-  **Menu Management** - CRUD operasi menu lengkap
-  **AI-Powered Recommendations** - Rekomendasi paket makanan dengan GROQ API
-  **Analytics Dashboard** - Analisis penjualan dan performa
-  **Customer Feedback** - Manajemen feedback dengan sentiment analysis
-  **Stock Predictions** - Prediksi stok berbasis AI
-  **Promotion Generation** - Rekomendasi promosi otomatis
-  **Menu Trends** - Analisis tren menu

Setup & Konfigurasi

Environment Variables (.env)

```
env
```

```
# Database Configuration
MYSQL_HOST=localhost
MYSQL_PORT=8889
MYSQL_USER=root
MYSQL_PASSWORD=root
MYSQL_DATABASE=restomate

# GROQ API Configuration
GROQ_API_KEY=gsk_dBZ6yilEvm0ROZrhcRg4WGdyb3FYMIltgrwErrXyu0XcY0ztxnBAI

# Next.js Configuration
NEXTAUTH_SECRET=your_nextauth_secret_here
NEXTAUTH_URL=http://localhost:3000
```

Dependencies

```
json

{
  "mysql2": "^3.6.0",
  "next": "^14.0.0",
  "clsx": "^2.0.0",
  "tailwind-merge": "^2.0.0"
}
```

GROQ Model Update

⚠ **PENTING:** Model `llama-3.1-70b-versatile` sudah didecommissioned. Semua API sekarang menggunakan `llama3-8b-8192`.

Struktur Database

Tabel Utama:

```
sql
```

-- *Restoran*

RESTAURANT (id_restaurant, email, password)

-- *Menu Items*

menu (Id_Menu, Nama_Menu, Harga, Kategori, Deskripsi, id_restaurant)

-- *Customer Orders*

Customer (Invoice_Id, Tanggal_Order, Harga_Total, id_restaurant)

-- *Order Items*

MEMESAN_MENU (id_menu, kuantitas, id_customer)

-- *Food Packages*

PAKET (id_paket, id_menu, id_restaurant)

-- *Stock Management*

STOK (id_stok, nama_bahan, kuantitas, tanggal_pembelian, tanggal_exp, id_menu, id_restaurant)

-- *Customer Feedback*

CUSTOMER_FEEDBACK (id_feedback, id_customer, id_restaurant, rating, feedback_text, feedback_date)

API Endpoints

Menu Management (/api/menu)

GET - Fetch Menu Items & Food Packs

http

GET /api/menu?restaurant_id=1

Response:

json

```
{
  "success": true,
  "data": {
    "menuItems": [...],
    "foodPacks": [...],
    "summary": {
      "totalMenuItems": 25,
      "totalFoodPacks": 10,
      "categories": ["Makanan Utama", "Minuman", "Snack"],
      "priceRange": { "min": 5000, "max": 50000 }
    }
  }
}
```

POST - Add New Menu Item

```
http

POST /api/menu
Content-Type: application/json

{
  "name": "Nasi Gudeg Special",
  "price": 25000,
  "category": "Makanan Utama",
  "description": "Gudeg khas Yogyakarta dengan ayam",
  "restaurant_id": 1
}
```

PUT - Update Menu Item

```
http

PUT /api/menu
Content-Type: application/json

{
  "id": 1,
  "name": "Nasi Gudeg Premium",
  "price": 30000,
  "category": "Makanan Utama",
  "description": "Updated description"
}
```

DELETE - Remove Menu Item

http

DELETE /api/menu?id=1

 AI Food Pack Recommendations (`/api/menu/food-packs`)

GET - Generate AI Recommendations

http

GET /api/menu/food-packs?restaurant_id=1&type=recommendations

Response:

json

```
{
  "success": true,
  "data": {
    "packs": [
      {
        "id": "ai_pack_1234567890_0",
        "name": "Paket Hemat Spesial",
        "description": "Kombinasi strategis menu terpilih",
        "items": ["Nasi Gudeg", "Es Teh Manis"],
        "price": 27000,
        "originalPrice": 30000,
        "discountPercent": 10,
        "reasoning": "Paket ini menggabungkan makanan utama populer",
        "estimatedDemand": "High",
        "category": "Value Pack",
        "generated": true
      }
    ]
  }
}
```

Features:

-  **Retry Mechanism:** 3 attempts sebelum fallback
-  **LLM Analysis:** Menggunakan data penjualan real
-  **Data-Driven:** Basis rekomendasi dari performa menu
-  **Fallback System:** Enhanced algorithmic generation

GET - Comprehensive Analytics

http

GET `/api/analytics?restaurant_id=1&period=30&start_date=2025-01-01&end_date=2025-01-31`

Query Parameters:

- `restaurant_id`: ID restoran (default: 1)
- `period`: Periode hari (default: 30)
- `start_date`: Tanggal mulai (optional)
- `end_date`: Tanggal akhir (optional)

Response Data:

json

```
{
  "success": true,
  "data": {
    "totalRevenue": 15750000,
    "totalOrders": 847,
    "averageOrderValue": 18599,
    "topSellingItems": [...],
    "revenueByCategory": [...],
    "dailySales": [...],
    "monthlySales": [...],
    "customerInsights": {
      "uniqueCustomers": 523,
      "avgOrderValue": 18599,
      "ordersPerCustomer": 1.62
    },
    "inventoryStatus": [...]
  }
}
```

Customer Feedback (`/api/feedback`)

GET - Fetch Feedback with Analysis

http

GET `/api/feedback?restaurant_id=1&page=1&limit=20&rating=5&period=30`

Features:

- 🗨️ **Sentiment Analysis:** Otomatis menganalisis sentimen feedback
- 📈 **Trend Analysis:** Tracking rating bulanan
- 🔍 **Advanced Filtering:** Filter by rating, status, periode
- 📊 **Summary Statistics:** Rating distribution dan insights

POST - Add New Feedback

http

POST /api/feedback

Content-Type: application/json

```
{
  "customer_id": 123,
  "restaurant_id": 1,
  "rating": 5,
  "feedback_text": "Makanan sangat enak dan pelayanan ramah!",
  "status": "pending"
}
```

📦 Orders Management (/api/orders)

GET - Fetch Orders

http

GET /api/orders?restaurant_id=1&page=1&limit=20&include_items=true&period=30

Response Features:

- 📋 **Order Details:** Lengkap dengan item breakdown
- 📊 **Summary Stats:** Total orders, revenue, popular items
- 📈 **Today's Performance:** Orders dan revenue hari ini
- 🕒 **Recent Orders:** 5 order terakhir

POST - Create New Order

http

POST /api/orders

Content-Type: application/json

```
{
  "restaurant_id": 1,
  "items": [
    { "menu_id": 1, "quantity": 2 },
    { "menu_id": 3, "quantity": 1 }
  ],
  "total_amount": 75000,
  "customer_info": {...}
}
```

AI Promotions (/api/promotions)

GET - Generate Promotion Recommendations

http

GET /api/promotions?restaurant_id=1&type=generate

AI Analysis Meliputi:

-  Menu performance data
-  Customer feedback analysis
-  Revenue optimization strategies
-  Market trends consideration

Response:

json


```
{
  "success": true,
  "data": {
    "promotions": [
      {
        "type": "Bundle Deal",
        "description": "Paket hemat menu populer + menu kurang laris",
        "reasoning": "Meningkatkan penjualan menu underperforming",
        "estimatedImpact": "+25% sales",
        "details": "Diskon 20% untuk pembelian paket"
      }
    ]
  }
}
```

POST - Apply Promotion

http

POST /api/promotions

Content-Type: application/json

```
{
  "promotion": {
    "type": "Happy Hour",
    "description": "Diskon 30% minuman jam 14-16",
    "reasoning": "Increase traffic during slow hours"
  },
  "restaurantId": 1
}
```

Menu Trends Analysis (/api/menu-trends)

GET - AI-Powered Trend Analysis

http

GET /api/menu-trends?restaurant_id=1

Response:

json

```
{
  "success": true,
  "data": {
    "trends": [
      {
        "trend": "rising",
        "itemName": "Nasi Gudeg",
        "currentSales": 150,
        "predictedSales": 180,
        "growthRate": 20,
        "confidence": 85,
        "reasoning": "Menu dengan performa terbaik",
        "recommendations": [
          "Pertahankan kualitas dan konsistensi",
          "Pertimbangkan variasi atau bundle deals"
        ],
        "category": "Makanan Utama",
        "seasonality": "Stabil sepanjang tahun"
      }
    ]
  }
}
```

Stock Predictions (`/api/generate-stock-predictions`)

GET - AI Stock Predictions

http

GET `/api/generate-stock-predictions?restaurant_id=1&period=week`

Parameters:

- `period`: `week` | `month` | `quarter`

Features:

-  **Pure LLM Analysis:** Menggunakan GROQ AI
-  **Comprehensive Data:** All-time stock dan sales data
-  **Period-Aware:** Analisis sesuai periode yang dipilih
-  **Actionable Insights:** Rekomendasi konkret

LLM Integration

GROQ API Configuration

typescript

```
// lib/llm.ts
const GROQ_API_URL = 'https://api.groq.com/openai/v1/chat/completions';

const response = await fetch(GROQ_API_URL, {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${process.env.GROQ_API_KEY}`,
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({
    model: 'llama3-8b-8192', // Updated model
    messages: [...],
    temperature: 0.7,
    max_tokens: 1500
  })
});
```

Fallback System

typescript

```
// Enhanced Fallback Strategy
try {
  // Try LLM first (3 attempts)
  const llmResult = await generateLLMRecommendations();
  return llmResult;
} catch (error) {
  // Fallback to algorithmic analysis
  return generateAlgorithmicFallback();
}
```

Rate Limiting

typescript

```
class SimpleRateLimiter {
  private lastRequest = 0;
  private minDelay = 3000; // 3 seconds

  canMakeRequest(): boolean {
    const now = Date.now();
    return (now - this.lastRequest) >= this.minDelay;
  }
}
```

Error Handling

Standard Error Response

```
json

{
  "success": false,
  "error": "Error type",
  "message": "Detailed error message",
  "data": null
}
```

HTTP Status Codes

- `200` - Success
- `400` - Bad Request (validation errors)
- `404` - Not Found (no data)
- `500` - Internal Server Error

Logging System

```
typescript

// Comprehensive logging
console.log('🔄 Fetching menu data...');
console.log('✅ Success: Retrieved 25 items!');
console.error('❌ Error:', error.message);
console.warn('⚠️ Warning: Fallback used');
```

Testing

Environment Setup

```
bash
```

```
# 1. Clone repository
```

```
git clone <repository-url>
```

```
# 2. Install dependencies
```

```
npm install
```

```
# 3. Setup database
```

```
mysql -u root -p restomate < scripts/restomate.sql
```

```
# 4. Configure environment
```

```
cp .env.example .env
```

```
# Edit .env with your configurations
```

```
# 5. Run development server
```

```
npm run dev
```

API Testing

```
bash
```

```
# Test menu endpoint
```

```
curl -X GET "http://localhost:3000/api/menu?restaurant_id=1"
```

```
# Test AI recommendations
```

```
curl -X GET "http://localhost:3000/api/menu/food-packs?restaurant_id=1&type=recommendations"
```

```
# Test analytics
```

```
curl -X GET "http://localhost:3000/api/analytics?restaurant_id=1&period=30"
```

Database Verification

```
sql
```

```
-- Check data availability
SELECT COUNT(*) FROM Customer WHERE id_restaurant = 1;
SELECT COUNT(*) FROM menu WHERE id_restaurant = 1;
SELECT COUNT(*) FROM STOK WHERE id_restaurant = 1;

-- Verify relationships
SELECT c.Invoice_Id, c.Harga_Total, m>Nama_Menu
FROM Customer c
JOIN MEMESAN_MENU mm ON c.Invoice_Id = mm.id_customer
JOIN menu m ON mm.id_menu = m.Id_Menu
WHERE c.id_restaurant = 1
LIMIT 5;
```

Production Checklist

Security

- ☐ API key protection
- ☐ Input validation
- ☐ SQL injection prevention
- ☐ Rate limiting implementation

Performance

- ☐ Database indexing
- ☐ Query optimization
- ☐ Caching strategy
- ☐ Connection pooling

Monitoring

- ☐ Error tracking
- ☐ Performance metrics
- ☐ API usage analytics
- ☐ Health checks

Deployment

- ☐ Environment variables secured
- ☐ Database migrations
- ☐ SSL certificates
- ☐ Load balancing

Support & Maintenance

Regular Tasks

- 1. **Monthly:** Review API usage dan performance
- 2. **Weekly:** Check error logs dan fallback frequency
- 3. **Daily:** Monitor database performance
- 4. **Real-time:** Alert system untuk critical errors

Troubleshooting Common Issues

GROQ API Errors

```
bash

# Check API key
echo $GROQ_API_KEY

# Test connectivity
curl -H "Authorization: Bearer $GROQ_API_KEY" \
https://api.groq.com/openai/v1/models
```

Database Connection Issues

```
bash

# Test connection
mysql -h localhost -P 8889 -u root -p restomate -e "SELECT 1"

# Check running processes
ps aux | grep mysql
```

Performance Issues

```
sql

-- Check slow queries
SHOW PROCESSLIST;

-- Analyze table performance
EXPLAIN SELECT * FROM Customer WHERE id_restaurant = 1;
```